



Article



George 
@odysseus0z



Harness Engineering Is Cybernetics

 23  294  1.2K  704K  

Reading OpenAI's [harness engineering post](#), I kept having a feeling I couldn't place. Then it clicked: I'd seen this pattern before. Not once — three times.

The first was [James Watt's centrifugal governor](#) in the 1780s. Before it, a worker stood next to the steam engine adjusting the valve by hand. After it, a weighted flyball mechanism sensed rotational speed and adjusted the valve automatically. The worker didn't disappear. The job changed: from turning the valve to designing the governor.

The second was [Kubernetes](#). You declare desired state — three replicas, this image, these resource limits. A controller continuously observes actual state. When they diverge, the controller reconciles: restarts

New to X?

Sign up now to get your own personalized timeline!

 Sign up with Google

 Sign up with Apple

Create account

By signing up, you agree to the [Terms of Service](#) and [Privacy Policy](#), including [Cookie Use](#).

What's happening

Sports · Trending

#LIVvGS

Trending in Germany

Lukas

Fashion competition · Trending

#gntm

Sports · Trending

#BAYATA

[Show more](#)

[Terms of Service](#) | [Privacy Policy](#) | [Cookie Policy](#) | [MStV Transparenzangaben](#) | [Imprint](#) | [Accessibility](#) | [Ads info](#) | [More ...](#) © 2026 X Corp.

Don't miss what's happening

People on X are the first to know.

[Log in](#)

[Sign up](#)



engineer's job shifted from restarting services to writing the spec the system reconciles against.

The third is now. OpenAI describes engineers who no longer write code. Instead they design environments, build feedback loops, and codify architectural constraints — then agents write the code. [A million lines in five months](#), zero written by hand. They call it "harness engineering."

Same pattern each time. Norbert Wiener named it in 1948: cybernetics, from the Greek κυβερνήτης — steersman. The same root that gave Kubernetes its name. You stop turning the valve. You steer.

Each time the pattern appears, it's because someone built a sensor and actuator powerful enough to close the loop at that layer.

Why the codebase was the holdout

The codebase had feedback loops, but only at the lower levels. Compilers close a loop on syntax. Test suites close a loop on behavior. Linters close a loop on style. These are real cybernetic controls — but they only operate on properties that can be checked mechanically. Does it compile? Does it pass? Does it follow the rules?

Everything above that — does this change fit the system's architecture? is this the right approach? is this abstraction going to cause problems as the codebase grows? — had no sensor and no actuator. Only humans could operate at that level, on both sides: judging quality and writing the fix.

LLMs changed both at once. They can sense at the level humans used to own — and act at the same level: restructure a module, redesign an inconsistent interface, rewrite a test suite around the contracts that actually matter. For the first time, the feedback loop can close where the important decisions are made.

But closing the loop is necessary, not sufficient. Watt's governor needed to be tuned. Kubernetes controllers need the right spec. And an LLM working on your codebase needs something harder to provide.

New to X?

Sign up now to get your own personalized timeline!



Sign up with Google



Sign up with Apple

Create account

By signing up, you agree to the [Terms of Service](#) and [Privacy Policy](#), including [Cookie Use](#).

What's happening

Sports · Trending

#LIVvGS

Trending in Germany

Lukas

Fashion competition · Trending

#gntm

Sports · Trending

#BAYATA

[Show more](#)

[Terms of Service](#) | [Privacy Policy](#) | [Cookie Policy](#) |

[MStV Transparenzangaben](#) | [Imprint](#) | [Accessibility](#) |

[Ads info](#) | [More ...](#) | © 2026 X Corp.

Don't miss what's happening

People on X are the first to know.

[Log in](#)

[Sign up](#)



Calibrating the sensor and actuator

Getting the basic feedback loop working — tests that agents can run, CI that gives parseable output, error messages that point to the fix — is table stakes. [Carlini demonstrated this](#) when he had 16 parallel agents build a C compiler: [embarrassingly simple prompts](#), but carefully designed test infrastructure. "Most of my effort went into designing the environment around Claude — the tests, the environment, the feedback."

The harder problem is calibrating the sensor and actuator with knowledge specific to *your* system. This is where most people get stuck, and where they blame the agent.

"It keeps doing the wrong thing. It doesn't understand our codebase." The diagnosis is almost always wrong. The agent isn't failing because it lacks capability. It's failing because the knowledge it needs — what "good" means for your system, which patterns your architecture rewards, which it avoids — is locked inside your head, and you haven't externalized it. Agents don't learn through osmosis. If you don't write it down, the agent makes the same mistakes on the hundredth run as the first.

The work is making your judgment machine-readable. Architecture docs that describe actual layering and dependency direction. Custom linters with remediation instructions baked in. Golden principles that encode your team's taste. OpenAI [found exactly this](#): they spent 20% of every Friday cleaning up "AI slop" — until they encoded their standards into the harness itself.

The only way forward

The practices this demands — documentation, automated testing, codified architectural decisions, fast feedback loops — were always correct. Every engineering book written in the last thirty years recommends them. Most people skip them because the cost of skipping was slow and diffuse: gradual quality decline, painful onboarding, tech debt that compounds quietly.

Agentic engineering makes the cost extreme. Skip the documentation and the agent ignores your conventions — not on one PR, but on every PR, at machine speed, around the clock. Skip the tests and the feedback loop can't close at all. Skip the architectural constraints and drift compounds faster than you can fix it. And here's the trap: you can't use agents to clean up the mess if the agents don't know what clean looks like. Without the calibration, the machines that created the problem can't solve it either.

The practices haven't changed. The penalty for ignoring them has become unbearable.

Don't miss what's happening

People on X are the first to know.

New to X?

Sign up now to get your own personalized timeline!

 Sign up with Google

 Sign up with Apple

Create account

By signing up, you agree to the [Terms of Service](#) and [Privacy Policy](#), including [Cookie Use](#).

What's happening

Sports · Trending

#LIVvGS

Trending in Germany

Lukas

Fashion competition · Trending

#gntm

Sports · Trending

#BAYATA

[Show more](#)

[Terms of Service](#) | [Privacy Policy](#) | [Cookie Policy](#) |

[MStV Transparenzangaben](#) | [Imprint](#) | [Accessibility](#) |

[Ads info](#) | [More ...](#) | © 2026 X Corp.

[Log in](#)

[Sign up](#)



The generation-verification asymmetry — the intuition behind P vs NP, demonstrated empirically for LLMs by Cobbe et al. — points to where this goes. Generating a correct solution is harder than verifying one. You don't need to out-implement the machine. You need to out-evaluate it: specify what "correct" looks like, recognize when the output misses, judge whether the direction is right.

The workers who designed Watt's governor didn't go back to turning valves. Not because they couldn't. Because it no longer made sense.

Want to publish your own Article?

[Upgrade to Premium](#)

11:54 PM · Mar 7, 2026 · **704.5K** Views

New to X?

Sign up now to get your own personalized timeline!

Sign up with Google

Sign up with Apple

Create account

By signing up, you agree to the [Terms of Service](#) and [Privacy Policy](#), including [Cookie Use](#).

What's happening

- Sports · Trending

#LIVvGS
- Trending in Germany

Lukas
- Fashion competition · Trending

#gntm
- Sports · Trending

#BAYATA
- Show more

[Terms of Service](#) | [Privacy Policy](#) | [Cookie Policy](#) | [MStV Transparenzangaben](#) | [Imprint](#) | [Accessibility](#) | [Ads info](#) | [More ...](#) | © 2026 X Corp.